

School of EEE
IC4004_IC4003_IC4001 Internship

Written Assignment
Diploma in Computer Engineering
Semester 2, 2025/26

Prepared For: Soo Bee Hong

Prepared By: Dion Choy

Class: DCPE/FT/3B02

Date: 4/2/2026

Abstract

This report documents a 22-week internship at the Defence Science and Technology Agency within the Advanced Systems Programme Centre, Counter-Unmanned Aerial Systems, from 15 September 2025 to 13 February 2026. The internship focused on radio-frequency spectrum sensing in Singapore's congested urban environment, with the objective of detecting anomalous, and particularly drone related signals, on the 2.4GHz and 5.8GHz bands. It discusses possible solutions to the problem such as machine learning models for anomaly detection. The report also included contributions to an RF Signal Detector PCB and other tasks like field testing and observing trials of equipment. Additionally, it mentions skills and knowledge learnt and acquired through the internship.

Acknowledgement

My heartfelt thanks go to DSTA, and specifically the Counter-Unmanned Aerial Systems cluster under the Advanced Systems Programme Centre, for hosting me and providing a suitable environment for me to apply and expand my skills in computer engineering and machine learning and learn new skills like RF engineering.

I am grateful to my supervisor, Waylon Ng, for his guidance and insights through my internship. His support and mentorship was crucial to my learning and development, giving appropriate feedback and criticisms throughout the project to strengthen my understanding.

I would like to express my sincere appreciation of my mentor, Calvin Wen, for imparting technical knowledge important for the success of the project. The guidance helped me build a stronger foundation in the relevant fields, shaping how I approach the project's main problem.

I would also like to thank full-time staff, Tan Jian Qing and Lai Zi Iun, for collaborating with me, giving feedback, and support during this internship. They gave me the room to grow and learn new skills, educating me through their own projects.

Besides DSTA, I am also grateful to my Liaison Officer, Ms Soo Bee Hong, for her support through the internship period. Her guidance in school-related administrative matters allowed me to have a seamless internship experience.

Finally, I am thankful to Singapore Polytechnic for not only the opportunity to intern at DSTA, but the education provided over the past three years. It has truly been a rewarding experience where I had the chance to learn important skills for the working world and interact with a multitude of people.

Table of Contents

Abstract.....	2
Acknowledgement.....	2
1. Introduction.....	5
1.1. Name, History & Background of the DSTA.....	5
1.2. Internship Details.....	5
2. About the Company.....	5
2.1. Company Culture.....	5
2.2. Organizational Structure.....	6
2.3. Workflow and collaboration.....	6
2.4. Interesting features of the company.....	7
3. About the Industry.....	7
3.1. How the Industry is Organized.....	7
3.2. Careers in the Industry.....	7
3.2.1. Career Prospects - in the Company and in the Industry Sector.....	7
3.2.2. Recruitment and Career Progression.....	8
3.2.3. Attractiveness of Company.....	8
4. My Work Experience (as an Intern).....	8
4.1. The Intern's Role and how it fits in the organization.....	8
4.2. The Project – CDIO Stages.....	8
4.2.1. Problem Statement (Conceive).....	8
4.2.2. Possible Solutions (Design).....	9
4.2.3. Implement.....	12
4.2.4. Operate.....	24
4.3. Other tasks and projects.....	28
4.4. Skills Required and Company Expectations.....	30
4.5. Any other Interesting notes about the Internship.....	31
5. Reflection and Learning Points.....	31
5.1. Reflection on CDIO Skills.....	31
6. Recommendations for the Company.....	32
6.1 Learning Points that benefit the company.....	32
6.2 Recommendations Made and Contributions to the company.....	32
6.3 What can the Company do to remain competitive.....	32
7. Careers in the Industry.....	33
7.1 My Own Lessons Learnt.....	33
7.2 Suggestions and recommendations to better prepare students.....	33

8. Conclusion.....	33
References.....	34
Appendix.....	35

1. Introduction

1.1. Name, History & Background of the DSTA

The Defence Science and Technology Agency (DSTA) is a technology organization crucial to the security and defence of Singapore. It supports the Singapore Armed Forces (SAF) and Singapore Ministry of Defence (MINDEF) through the acquisition, development, and integration of systems.



Fig. 1.1.1: DSTA's Logo

DSTA was founded on 15 March 2000 through the combination of the Defence Technology Group, the Systems and Computer Organization, and the Defence Medical Research Institute [1].

1.2. Internship Details

The internship at DSTA lasted from 15 September 2025 to 13 February 2026 where I was assigned to the Advanced Systems Programme Centre (ASPC), under the Counter-Unmanned Aerial Systems (C-UAS) cluster. Over the course of 22 weeks, I interfaced with Software-Defined Radios (SDRs) to perform analysis on Radio Frequency (RF) data, in addition to assisting with testing of RF-related technologies.



Fig 1.2.1: Advanced Systems Logo

2. About the Company

2.1. Company Culture

Full-timers contribute to Singapore's defence capabilities through meaningful work. It provides fulfilling opportunities for staff to work on the latest emerging technologies for defence purposes. Technologies such as AI, machine learning, and mixed reality portals are used to solve operational problems and challenges. All of this is supported through DSTA Academy, a comprehensive list of resources to enrich staff and improve both technical and managerial skills.

Apart from interesting work, there is a large emphasis on teamwork. New hires go through a mentorship programme, where more experienced colleagues guide and mentor new hires. Colleagues are also encouraged to eat both within and outside of their Programme Centres to build a more bonded and unified company.

2.2. Organizational Structure

Within DSTA, operation is split into Programme Centres (PCs) which are the primary units responsible for delivering programmes and providing effective systems (Fig. 2.2.1) [2]. DSTA also works closely with DSO National Laboratories, and Centre for Strategic Infocomm Technologies (CSIT). These organizations are the leading agencies of Singapore's Defence Technology Community (DTC) [3].



Fig. 2.2.1: DSTA's Organizational Chart

2.3. Workflow and collaboration

The delivery of projects within the DSTA workplace requires cross-domain competencies to integrate systems together. DSTA performs system engineering, creating a system of systems where hardware and software are developed individually first and integrated next.

Projects first start with DSTA Masterplanning and Systems Architecting, where they perform front-end planning to identify potential pain points and determine how SAF and the defence can be improved. The required resources and contracts are then signed through Procurement.

Each PC specializes in their own area, with engineering-related Air, Land, and Naval Systems to procure related technology. Other PCs take up crucial roles such as sensors and electronics in Advanced Systems, and system infrastructure and power in Building and Infrastructure.

All of this is supported by the digital side with C3 Development, Digital Hub, and Information. Cybersecurity supports them by keeping the digital systems safe, while InfoComm Infrastructure focuses on networking and connecting digital solutions together.

The systems are finally integrated with real-world systems and networks and deployed through Systems Engineering and C3 Centre. Meanwhile, Systems Management ensures the continued support of deployed systems.

2.4. Interesting features of the company

DSTA frequently takes part in community service through events such as monthly donations with Community Chest, and the annual Boys' Brigade Share-A-Gift programme. Additionally, DSTA is committed to environmental sustainability, with efforts in line with Singapore Green Plan 2030. This is expressed through their three key thrusts [4]:

1. We adopt and embed environmental sustainability policies and practices into our core business areas, to promote sustainable operations and responsible resource management.
2. We harness technology to enhance efficiency and drive innovative solutions to minimize resource consumption.
3. We foster a culture of environmental sustainability by engaging our employees and stakeholders in our environmental initiatives and encouraging responsible practices across the organization.

3. About the Industry

3.1. How the Industry is Organized

The main customers are mainly SAF and Ministry of Defence (MINDEF), however, there are sometimes projects need to be developed for Whole-Of-Government as well. DSTA works with the government and industry to purchase and collaborate on products and technology required.

As DSTA assists with the defence of Singapore, there are no competitors and the whole of industry (DTC) works together to ensure the safety and security of everyone. The company's service is unique as the main goal is to develop defence technology. Hence, it stays relevant by keeping up with new technologies and identifies how it can be used to assist in the defence and national security needs of Singapore.

3.2. Careers in the Industry

3.2.1. Career Prospects - in the Company and in the Industry Sector

Within the Defence Technology Community, there are a numerous number of entities. Besides DSTA, other companies include DSO and CSIT, as well as entities within MINDEF like Future Systems and Technology Directorate (FSTD), Defence Technology Collaboration Office (DTCO), etc. The DTC continues to advance technology to respond to uncertain threats while supporting the SAF.

Beyond the DTC, there are other companies in the industry which are frequently collaborated with. Such examples could include ST Engineering, Government Technology Agency of Singapore (GovTech Singapore), Home Team Science and Technology Agency (HTX), and more. All of these companies assist with the development of technology through engineering and digital means.

3.2.2. Recruitment and Career Progression

Within DSTA, there exists competency frameworks to increase technical skills. One such example is the Technology Graduates Programme (TGP) where graduating university students are encouraged to apply to DSTA. The TGP aims to build a strong foundation in one's early career through a structured 2-year development programme. Through this, one will be able to identify their interests and broaden horizons through technical skills, build and hone leadership skills, connect and network with like-minded peers, and gain hands-on experience through projects at DSTA.

Over the years, staff will be able to progress through various roles, starting out as an Engineer and advancing to Senior Engineer. After Senior Engineer, staff can choose between Principal Engineer and Programme Manager and advance to the Senior Principal Engineer and Senior Programme Manager respectively. Finally, if the staff express exemplary skills, conduct, and passion, they can move on to Head of Engineering and Head of Department respectively. Apart from roles within each PC, those leading DSTA as a whole are the Deputy Director, Director, Deputy Chief Executive, and Chief Executive.

3.2.3. Attractiveness of Company

Besides the TGP to enrich and deepen new employees, long-term hires also have the chance to go for courses to build leadership skills, allowing them to better mentor those below them. Apart from soft skills, DSTA Academy provides courses to build up technical skills as well. All of these constantly empower employees to improve their quality of work, contributing back to the company and country overall.

4. My Work Experience (as an Intern)

4.1. The Intern's Role and how it fits in the organization

To the organization, an intern's role is to learn as much as they can while enjoying their time at the company. Typically, due to security concerns, an intern's project has a layer of abstraction to protect restricted information from being exposed. However, I had the opportunity to work on projects with little restricted information. These projects contribute back to the company by providing software and hardware solutions to the identified problems.

Outside of developing and engineering solutions, I had to assist with testing of RF technologies. Some examples include verifying the frequency band which drones are transmitting on, range tests of transmitters, and the verification of hardware circuits. Additionally, I was brought onto various sites such as the Changi Air Base to learn how equipment is set up, as well as aid in the collection of RF data.

4.2. The Project – CDIO Stages

4.2.1. Problem Statement (Conceive)

Due to Singapore's urban landscape, the radio frequency spectrum has become more congested. To preserve the fidelity of secure communications for military use, there is a need

to ensure that malicious agents do not disrupt secure channels. Thus, machine learning can be used for spectrum sensing applications to identify the presence of anomalous signals.

An anomaly can be defined as an outlier which appears to be inconsistent with the remainder of the data. Within the context of spectrum sensing, anomalies can be malicious agents trying attempting to take control of communication channels through RF signals. In the context of C-UAS, drones are increasingly used in modern conflicts. These drones produce RF signals with unique signatures which are not typically present on the spectrum. Therefore, these signals can be used as indicators for the presence of drones. This will be the focus of the project; to detect anomalous radio signals with an emphasis on those transmitted by drones.

The frequencies of interest are the 2.4GHz band (2.400 to 2.4835GHz) and the 5.8GHz band (5.725 to 5.850GHz)

4.2.2. Possible Solutions (Design)

Several methods can be used to perform anomaly detection. Below are such methods:

- Statistical Methods
 - Z-Score
 - Interquartile Range (IQR)
- Machine Learning
 - One-Class SVM (Support Vector Machines)
 - Autoencoders
 - Object Detection

Statistical Methods

Statistical methods are methods which use data analysis, combined with threshold values, to identify outliers. Different methods use distinct formula for calculating the thresholds, where the two discussed will be Z-Score and IQR.

Z-Score is expressed by the formula,

$$z = \frac{x - \mu}{\sigma}$$

where Z represents how many standard deviations (σ) away the value (x) is from the mean (μ). Anomalies would be classified as data points where Z-Score is outside of the range $\mu \pm (\sigma * \text{threshold})$. The threshold can be adjusted, typically from around a value of 2 to 3.

In contrast, IQR is defined as the difference between the 75th percentile (Q_3) and 25th percentile (Q_1). Anomalies, also referred to as outliers, would be observations where the value is below $Q_1 - 1.5 * \text{IQR}$ or above $Q_3 + 1.5 * \text{IQR}$.

Comparing the two, Z-Score allows for a good model of data with a normal distribution. However, IQR works well with skewed data. In addition, the range of valid values is less affected by outliers as quartiles are used instead of the mean.

While both Z-Score and IQR are simple and fast to implement, both require prior knowledge of the distribution of data. This makes statistical methods sensitive to the input data and parameters, reducing robustness and flexibility for different environments. Furthermore, there is much information lost when representing RF signals with a single value (typically Received Signal Strength Index, RSSI) for analysis. Therefore, the attempted solutions will mainly involve the more robust machine learning methods.

Machine Learning

Machine learning methods are split into supervised and unsupervised learning. Unsupervised learning consists of One-Class SVM and Autoencoders (self-supervised) while supervised learning includes Object Detection.

In unsupervised learning, the dataset consists of only normal instances, i.e., WiFi, Bluetooth, and other expected RF signals. This removes the need for the dataset to be labeled as it is assumed that almost all instances are normal instances. Furthermore, due to the nature of anomalous data, it can be challenging to collect sufficient samples to make up a class large enough for labeling. Hence, this method can increase developer productivity by focusing on the model rather than data acquisition.

The goal of One-Class SVMs is to define a boundary around the majority class in the feature space, shown in Figure 4.2.1. This takes advantage of the assumption that anomalies deviate significantly from the usual patterns of normal instances. When the boundary is being determined, the margin between the normal and anomalous data points are maximized, allowing for a better separation of what is ordinary and unusual [5].

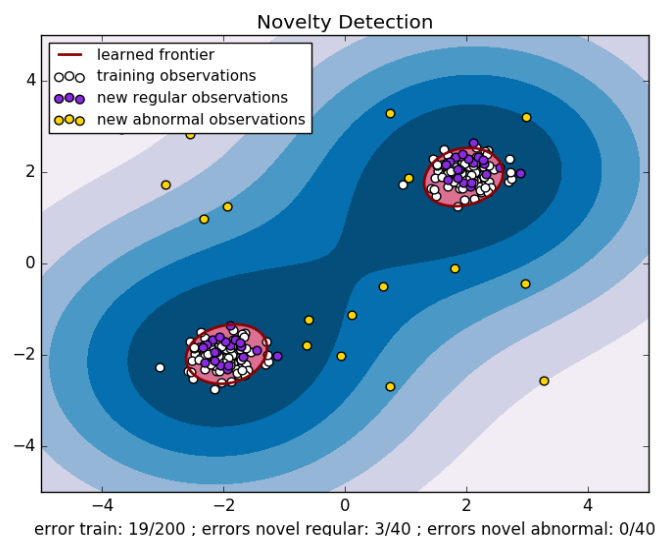


Fig. 4.2.1: Visualization of boundary (scikit-learn, 2025)

Generally, SVMs use a kernel to transform the input into a higher dimensional space. In the original dimensional space, instances might not be able to be separated linearly. Hence, by mapping the features to a higher dimension, a more optimal hyperplane can be generated. The hyperplane can be represented by the following equation [6]:

$$\mathbf{w}^T \mathbf{x} + b = 0$$

Where:

- $\mathbf{x}$ is the input feature vector,
- $\mathbf{w}$ is the normal vector to the hyperplane,
- b is the bias of the hyperplane.

Thus, with the hyperplane and learned normalcy region, One-Class SVMs detect, and flag instances which fall outside the boundary.

The second unsupervised learning method, Autoencoders work by compressing and reconstructing the input data. Autoencoders are made up of three main parts, the encoder, bottleneck, and decoder (Fig. 4.2.2). Input data is compressed using an encoder, which flows into the bottleneck layer, also known as the latent space. The latent space then goes through the decoder, attempting to reconstruct the original input as close as possible.

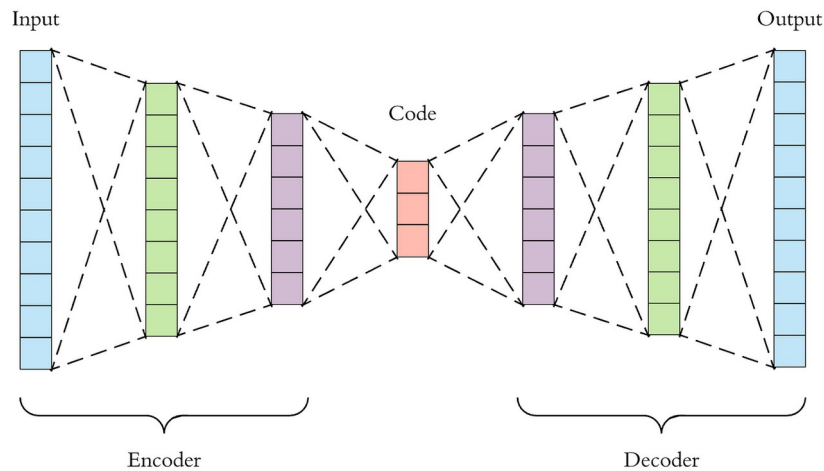


Fig.4.2.2: Architecture of an Autoencoder (Dertat, 2017)

When Autoencoders are trained exclusively on normal data, the model is trained to minimize the reconstruction error for normal instances. Thus, during the inference stage, if the input to the model is normal, the reconstruction error will be low. Conversely, when supplied with an anomalous input, the reconstruction error will be higher. A threshold can be set such that if the error crosses the threshold, the input will be flagged as anomalous.

There are several variations of Autoencoders such as the following:

- Convolutional Autoencoder (CAE)
- LSTM Autoencoder (LSTM-AE)

- Adversarial Autoencoder (AAE)
- (Convolutional) Variational Autoencoder (VAE/CVAE)

The different variations slightly differ but all have the same concepts of compressing and reconstructing the input. The individual differences will be expanded upon during the implementation phase.

The last method, object detection, is a supervised learning method which detects the presence of known signals. The signals present in the RF spectrum can be thought of as a set of normal and anomalous signals. Hence, by detecting the presence of known signals, the spectrum can be filtered out, leaving behind the unknown signals. This method is more akin to novelty detection, in contrast to typical object detection models being trained to identify anomalies, such as defective products.

Two main object detections models will be tested:

- You Only Look Once (YOLO)
- RoboFlow Detection Transformer (RF-DETR)

YOLO has a Convolutional Neural Network (CNN) based architecture, using convolutional layers and a custom C3K2 block to extract features. It also has a C2PSA block which uses an attention mechanism to focus on important areas.

Conversely, RF-DETR uses a Vision Transformer (ViT) based architecture and attention mechanisms. It uses the DINOv2 ViT backbone extract features from the input image, interleaving attention blocks to compromise between accuracy and latency.

4.2.3. Implement

All Machine Learning models will be implemented using the Python programming language. Each method requires different libraries, however, common libraries used include Numpy for data processing, and Matplotlib for data visualization, and OpenCV used for both aspects.

Besides data processing and visualization, there is a need to interface with Software-Defined Radios (SDRs). The first of such is Universal Software Radio Peripheral (USRP), a series of SDRs made by Ettus Research, which can be controlled using the USRP Hardware Driver (UHD) Python API. The second SDR commonly used is the ADALM-PLUTO (shortened to PlutoSDR), a low-cost SDR from Analog Devices. To interface with it, several drivers (libiio, libad9361-iio) and libraries (pyadi-iio) need to be installed.

All code and inference was performed on a T8 Firebat Pro/Plus mini computer. It uses an Intel N100 with 16 GB of RAM. Besides the T8 Firebat Pro/Plus, a portion of models were trained on a HP Zbook Studio G9 with a 64 GB of RAM and an NVIDIA GeForce RTX 3080 Ti Laptop GPU.

Software-Defined Radios

Before continuing with the implementation methodology, a brief explanation of SDRs will be given to provide easier understanding. In contrast to hardware based radios, SDRs consists of highly variable hardware and software, allowing the SDR to fit a larger number of needs. As there is a focus on collecting data, the receiving end of the SDR will be highlighted.

Typically, SDR receivers consist of the following components and subsystems (See Appendix A & B for architectures of USRP and PlutoSDR respectively):



Fig. 4.2.3: Typical architecture of receiver of an SDR

SDRs use a sampling method known as quadrature sampling, also known as complex sampling or IQ sampling. It works by sampling the same RF signal with both an “in-phase” and a 90-degrees out of phase component, called the “quadrature” component [7]. This makes use of the following mathematical identity:

$$\begin{aligned} A \cos(2\pi ft - \phi) \\ = I \cos(2\pi ft) + Q \sin(2\pi ft) \end{aligned}$$

where,

$$\begin{aligned} A &= \sqrt{I^2 + Q^2} \\ \phi &= \tan^{-1}\left(\frac{Q}{I}\right) \end{aligned}$$

allowing the amplitude and phase of a signal to be more easily represented with the in-phase and quadrature component. This returns a data stream made up of complex values, also known as IQ data. Another advantage of quadrature sampling is that the bandwidth that can be sampled is doubled. Due to the orthogonality of the in-phase and quadrature signals, both the amplitude and phase can be measured instead of direct sampling, where only the amplitude is measured.

This IQ data is the raw data collected before being processed for training the machine learning models.

Unsupervised Learning

Since both One-Class SVM and Autoencoders use unsupervised learning, the same dataset can be used. The dataset collected consists of 1500 instances of raw IQ data collected at the maximum bandwidth (56MHz) by the USRP SDR for 300,000 samples (~5.3ms). The data will be transformed as required for each model, with the most common being the Fast-Fourier Transform (FFT), and the Short-Time Fourier Transform (STFT) to produce a spectrogram.

The Fast Fourier Transform is an algorithm that returns the Discrete Fourier Transform (DFT) for a time-series input. The goal of a DFT is to transform the input from the time domain to the frequency domain, defined by the following formula [8]:

Let the time-series input be complex numbers $x_0, \dots, x_{N-1}$,

$$X_k = \sum_{n=0}^{N-1} x_n \cdot e^{-j2\pi \frac{k}{N}n} \quad \text{for } k=0, \dots, N-1$$

The transformation can be denoted by the following: $X = \mathcal{F}(x)$

The terms FFT and DFT are frequently used interchangeably. However, to be specific, FFT is the algorithm used to speed up calculation of the DFT. The most used algorithm is the Cooley-Tukey algorithm, also employed by the numpy library, but other algorithms exist such as the Prime-factor algorithm, Brunn's algorithm, Rader's algorithm, and more.

As the FFT returns a complex valued sequence of numbers, it needs to be converted into real-valued sequence for interpretation. Furthermore, the scaling of the signal can vary due to number of bins used in the FFT and the sampling rate of the data. As such, Power Spectral Density (PSD) can be used to solve both these issues. The FFT can be converted to PSD through the following formula [9]:

$$X_{PSD} = \frac{|X|^2}{f_s \cdot N}$$

where,

f_s = sampling frequency of SDR
 N = number of bins in FFT

Additionally, it may be useful to express the result as a logarithmic scale (dB/Hz) for better visualization. An example of the resulting PSD plot may be seen below (Fig. 4.2.4):

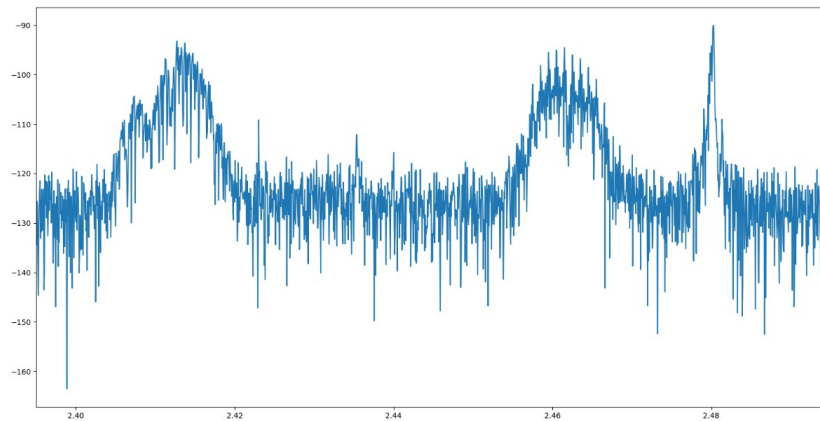


Fig. 4.2.4: Example PSD of 2.4GHz band

The Short-Time Fourier Transform splits the time signal into smaller segments and performs a FFT on each segment. Stacking up the output frequency domain signals result in a three-dimensional plot. The plot is usually represented two-dimensionally, with a time and frequency axis. The magnitude of each frequency represented by the colour intensity of pixels in the logarithmic scale. An example of the spectrogram can be seen below (Fig. 4.2.5):

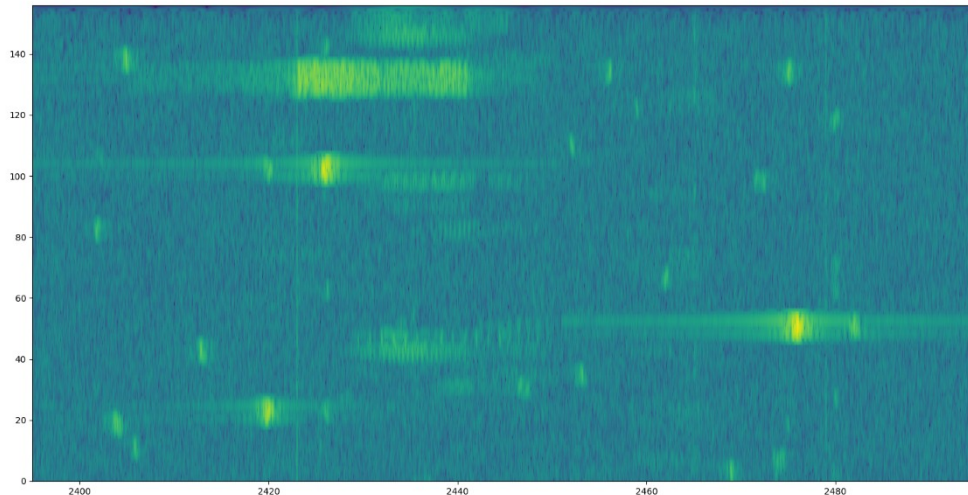


Fig. 4.2.5: Example spectrogram of 2.4GHz band

One-Class SVM

One-Class SVM uses the `OneClassSVM` class from `scikit-learn`.

For preprocessing of the raw IQ data, two methods were tested: FFT and spectrogram. The FFT remained the same as previously explained. However, as the One-Class SVM model only accepts a 1D feature vector, the resulting spectrogram has to be flattened. Both transformations are normalized using Z-Score Standardization, where $\mu=0$ and $\sigma=1$.

The model's hyperparameters include the kernel, ν , and γ . As previously explained, the kernel transforms the input data to allow better separation of features. Since the distribution of RF signal features is complex and likely non-linear, the best kernel would be to choose the Radial Basis Function (RBF), which is the most effective at capturing these relationships. ν describes the margin of the boundary generated by the model. The result is that the lower ν is, the larger the margin of error is, leading to fewer anomalies. Essentially, the smaller ν is, the more lenient the model is to anomalies. γ describes how complex the decision boundary is. The larger γ is, the more complex the boundary can be, allowing more intricate patterns to be identified.

Since the data is normalized, the input data's variance (σ^2) is equal to 1. Therefore, both "scale" and "auto" options for γ both have the same result. Hence, hyperparameter tuning only needs to be performed for ν .

One-Class SVM Results

Of the two preprocessing transformations (FFT and spectrogram), the spectrogram performed significantly better. This could be due to the fact that more information over time was available.

Below are results of the models after varying ν tested against a test dataset consisting of anomalies and normal signals (Table 4.2.1):

Nu	F-Score	Precision	Recall	Time
0.01	0.6829	0.8750	0.5600	0.382117
0.05	0.6707	0.8358	0.5600	0.386854
0.1	0.6588	0.8000	0.5600	0.396179
0.5	0.7458	0.6471	0.8800	1.101059

Table 4.2.1: Results of One-Class SVM models

As nu increases, the accuracy of models increases as seen by the F-Score. However, in the context of anomaly detection and C-UAS, it is best to reduce false positives. Therefore, the best metric to look at is precision, where $Precision = TP / (TP + FP)$. Ideally, precision should equal to 1, signifying there are no false positives present. Hence, Nu = 0.01 would give the best model for this use case, with the highest precision of 0.8750. Furthermore, it also has the second best F-Score of 0.6829

Autoencoders

Autoencoders will be implemented through the pytorch framework.

Unlike One-Class SVM, the spectrogram transformation of resized to (1024, 1024) will mainly be used as it can capture most information of signals against time. However, the FFT will be used for autoencoder architecture which is better suited for a one-dimensional input vector.

While training the model, different loss functions (also known as cost or error function) can be used. The most common one is the Mean Squared Error (MSE) function:

$$MSE = \frac{1}{n} \sum (Y_i - \hat{Y}_i)^2$$

where Y is the vector of observed values of the variable being predicted, and $\hat{Y}$ being the predicted values. It is used as it penalizes large errors due to the squaring of the error terms, making computations more efficient. If using MSE, inputs are normalized using Z-Score standardization.

The second loss function used is Structural Similarity Index Measure (SSIM). The function is taken from torchmetrics, a collection of metric implementations for PyTorch. SSIM has a general formula but the implementation uses the simplified formula:

$$SSIM(x, y) = \frac{(2\mu_x\mu_y + c_1)(2\sigma_{xy} + c_2)}{(\mu_x^2 + \mu_y^2 + c_1)(\sigma_x^2 + \sigma_y^2 + c_2)}$$

where,

μ =pixel sample mean
 σ^2 =sample variance
 σ =sample covariance
 $c_1=(k_1 L)^2, c_2=(k_2 L)^2$ to stabilize division
 L =dynamic range of pixel values
 $k_1=0.01, k_2=0.03$ by default

SSIM considers image degradation as a perceived change in structural information. It also differs from MSE by incorporating perceptual phenomena such as luminance masking and contrast masking instead of only comparing absolute errors [10]. If using SSIM, inputs are normalized using min-max scaling, changing the range from 0-1. Since SSIM returns a score from -1 (perfect anti-correlation) to 1 (perfect similarity), the formula has to be modified to 1-SSIM when used as a loss function for training. This is due to the model's goal of minimizing the error as much as possible.

To elaborate more on the architecture, Convolutional Autoencoders (CAEs) are autoencoders that extract features through convolution layers. In contrast to One-Class SVM, this method preserves hierarchical structure of the image. Furthermore, through parameter sharing of kernels, features such as edges or patterns can be identified regardless of the location in the image.

In CAEs, the input image is compressed using convolution and pooling layers while the image is reconstructed using transposed convolution layers (Fig. 4.2.6).

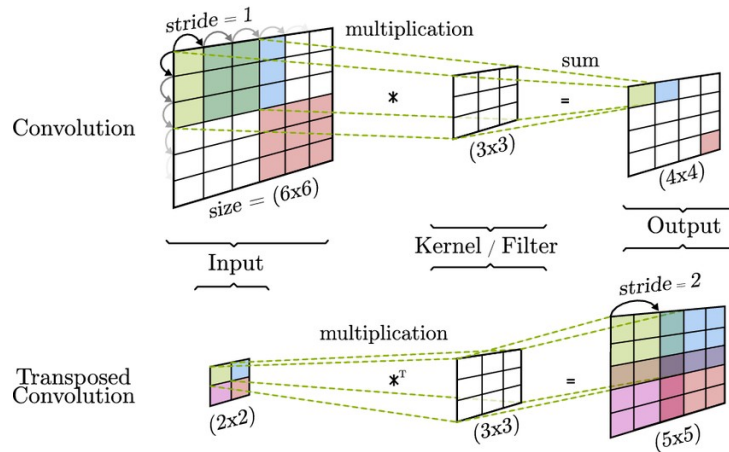


Fig. 4.2.6: Layers used in Convolutional Autoencoders

CAEs can be varied by changing the complexity of the neural network, i.e., number of encoder and decoder layers, and the number of filters on each layer.

Long-Short Term Memory Autoencoders (LSTM-AE) make use of Long-Short Term Memory (LSTMs), a special type of Recurrent Neural Network (RNNs). Similar to RNNs, LSTMs can retain information from previous inputs through the use of memory cells. The main difference is that LSTMs are more advanced, with an Input Gate, Forget Gate, and Output Gate to decide what information to store, remove, and pass respectively (Fig. 4.2.7).

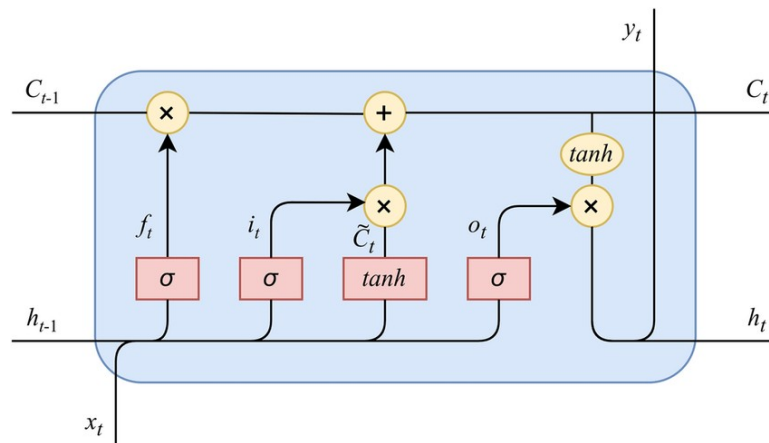


Fig. 4.2.7: Architecture of LSTM cell

In Figure 4.2.7, LSTMs take an input x_t , and return the hidden state h_t , and current cell state, C_t .

This allows the LSTM cells to remember important information while discarding irrelevant information over a longer period of time as compared to RNNs. Therefore, this architecture would supposedly be better at processing sequential or time-series data.

Adversarial Autoencoders (AAE) employ the use of a discriminator network to evaluate the autoencoder network. Normally, the latent space of the autoencoder is used for testing against the discriminator. However, this assumes that the distribution of normal data is known. Hence, instead of using the latent space, the implementation used will be like a Generative Adversarial Network (GAN). Like GANs, the output of the autoencoder will be used and compared against the original input. As such, the resulting architecture will look like Figure 4.2.8:

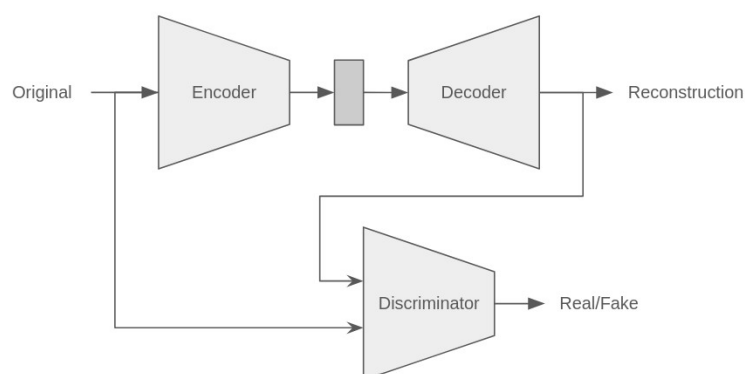


Fig. 4.2.8: Architecture of Adversarial Autoencoder

The autoencoder network and discriminator network “chase” each other to produce better reconstructions and classifications. Ideally, the discriminator would reach an accuracy of 0.5 while the autoencoder achieves a lower reconstruction error for normal data.

Lastly, the Variational Autoencoder (VAE) differs from the typical autoencoder by modeling the latent space as a probability distribution instead of fixed points. Both the encoder and decoder are similar to previous implementations of autoencoders. The main distinction is that the latent space get reparameterized through a learned mean and variance of the input data, which can be seen in Figure 4.2.9.

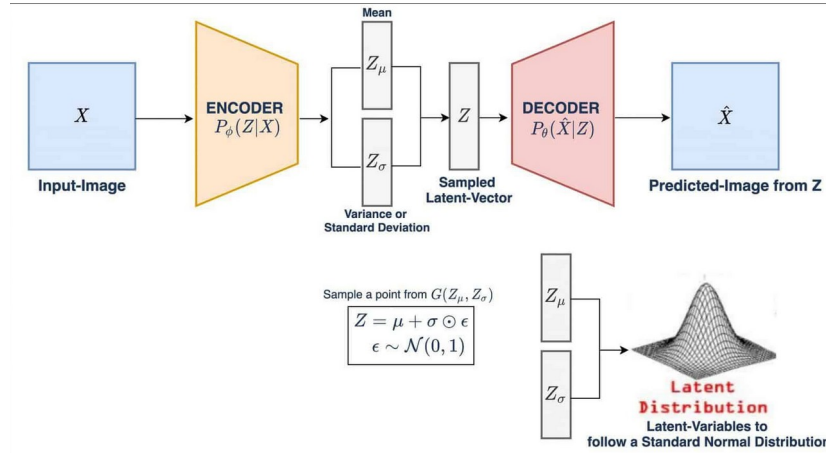


Fig. 4.2.9: Architecture of Variational Autoencoders

Due to the distribution, the resulting latent space is smooth and continuous. This allows the generation of realistic data samples, potentially leading to better accuracy. To model an image better, the encoder and decoders can be replaced with convolutional and transposed convolutional layers, creating a Convolutional Variational Autoencoder (CVAE).

Autoencoder Results

Starting with the LSTM-AE, the results were poor, with the reconstruction having little meaning. Both the raw IQ and spectrogram data were attempted. However, there was no significant improvement in the model performance. An example of the attempts can be seen in Figure 4.2.10 and 4.2.11:

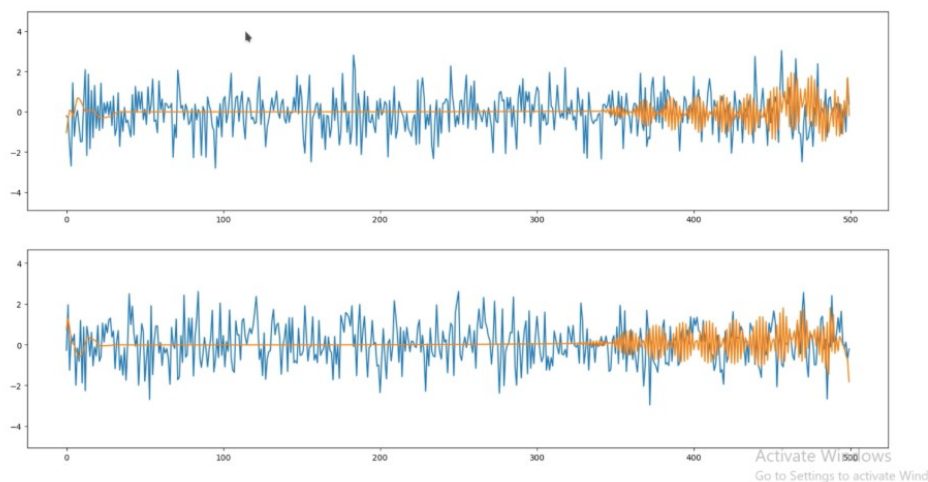


Fig. 4.2.10: Original (blue) and reconstruction (orange) of Real (top) and Imaginary (bottom) signals

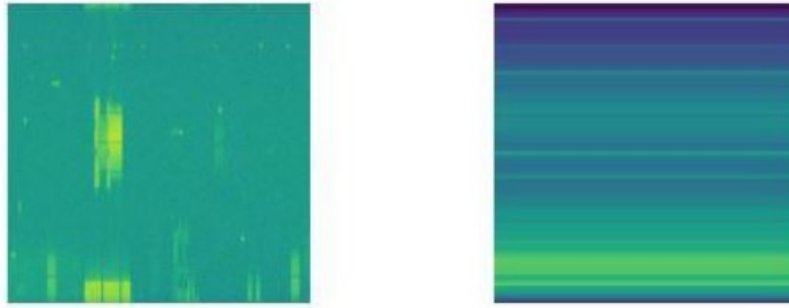


Fig. 4.2.11: Original (left) and reconstruction (right) of spectrogram

As seen, the reconstruction of the signals do not vary as much as the original. The reason for is could be that the patterns in the signal are too complex to be modeled well. This causes the models to “cheat” and produce the value which best reduces the error despite the low similarity.

Similarly, models that required flattening of the image performed poorly. One such example is the VAE model where it had a similar result to the models above, with no clear patterns or signals present in the reconstruction. Hence, focus of training moved to models with convolutional layers that maintain the hierarchical of the spectrogram image.

In Figure 4.2.12 are the remaining models and architectures tested:

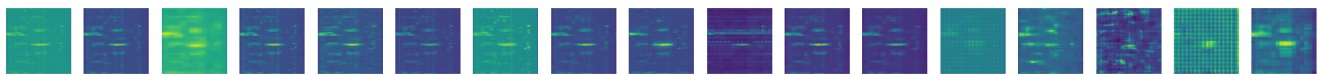


Fig. 4.2.12: Various models and reconstructions (original at leftmost)

Different types of architectures and loss functions were used to varying degrees of success. The following metrics are show below:

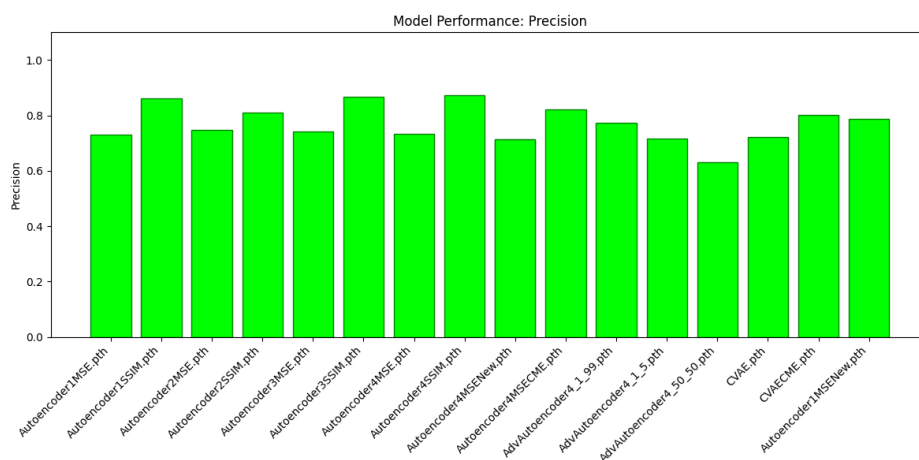


Fig. 4.2.13: Precision of models

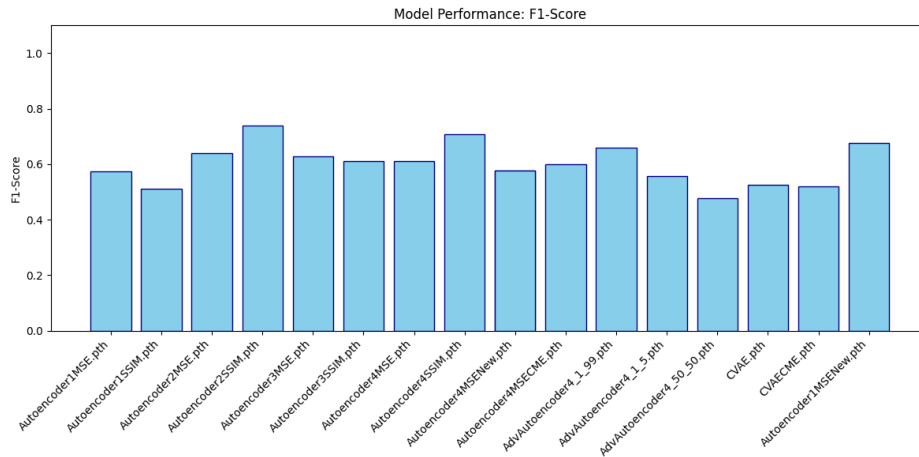


Fig. 4.2.14: F-Score of models

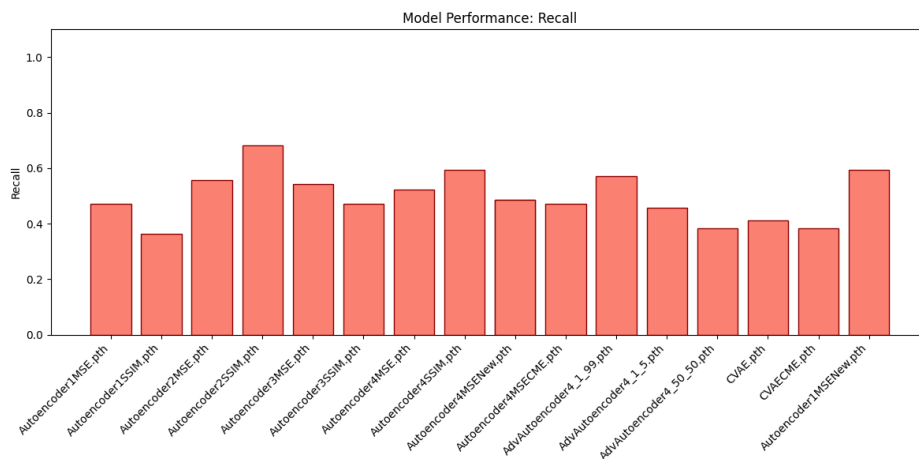


Fig. 4.2.15: Recall of models

Similar to One-Class SVM, false positives are discouraged. Therefore, the models will be selected using precision followed by F-Score and finally Recall. The top three models have been listed in Table 4.2 below, including the relevant metrics.

Model Name	Precision	F-Score	Recall	Time
Autoencoder4SSIM.pth	0.8723	0.7069	0.5942	0.349782
Autoencoder3SSIM.pth	0.8667	0.6103	0.4710	0.335901
Autoencoder1SSIM.pth	0.8621	0.5102	0.3623	0.384507

Table 4.2.2: Results of autoencoder models

The models listed all use the Convolutional Autoencoder Architecture with varying levels of complexity as well as the SSIM loss function (example of Autoencoder4 seen in Appendix C). Overall, all the models have a similar performance. However, based on the metrics discussed these three models will be the ones chosen for the operation stage.

Supervised Learning

Supervised Learning requires a labeled dataset for model training. The dataset from the journal “Spectrogram Data Set for Deep-Learning-Based RF Frame Detection” [11] will be used. It is a synthetic dataset that contains randomly generated Wi-Fi and Bluetooth signals, assumed to be the normal signals in the RF spectrum, in the YOLO format but can be converted to other formats if possible. The dataset also contains collisions between two signals. Since it refers to the collisions among known signals, the presence of the “collision” is of minor importance.

Both object detection models are intended to be used on images. While this causes the magnitude of some data to be lost, it should be sufficient for decent results to be used. As both models use publicly available implementations, training of models does not require much preprocessing work.

YOLO

The first object detection method experimented with YOLO models, using the pip ultralytics package for training and inference. As this is an object detection model, F-Score will be the main metric, in contrast to the anomaly detection methods which used precision.

Over different YOLO versions and sizes, the best one found was YOLOv11 medium with 20 epochs of training. Furthermore, from the loss curve (Figure 4.2.16), the train and validation loss follow each other well, showing that there is not much over-fitting.

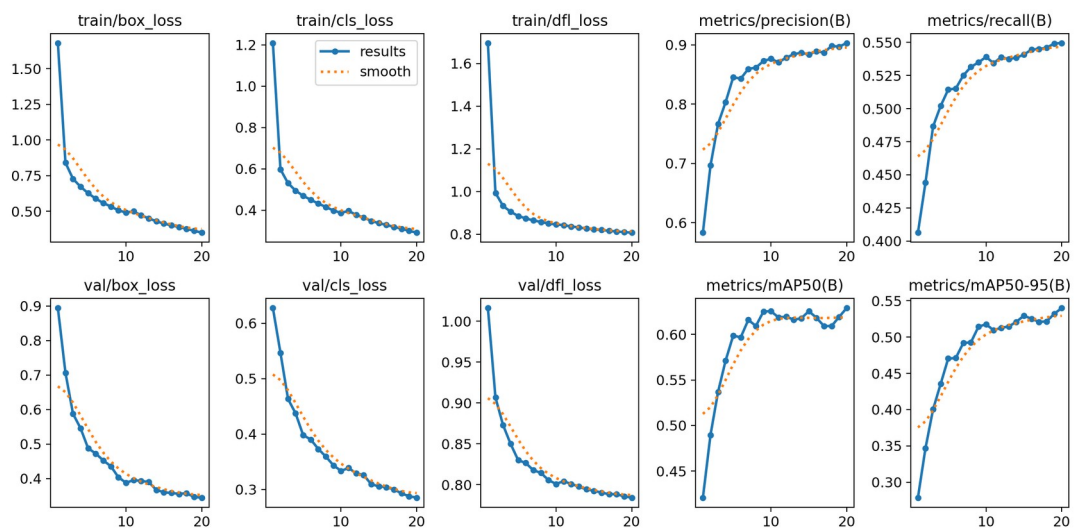


Fig. 4.2.16 Loss curves of YOLOv11m

It has the highest F-Score of 0.63 when the confidence threshold is 0.172 (Figure 4.2.17) and will be used in the operate stage.

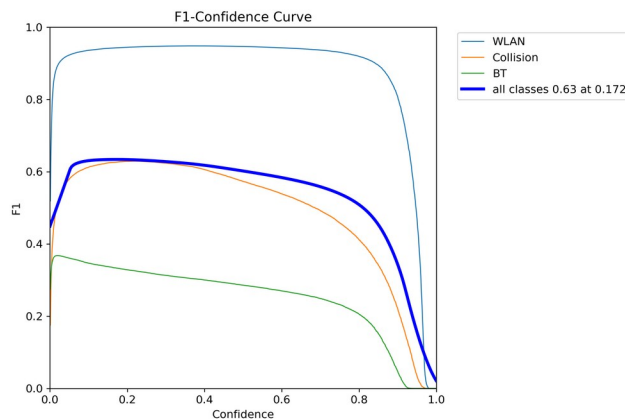


Fig. 4.2.17: F1-Confidence Curve of Autoencoder4

Besides F-Score, it has the highest mean average precision (mAP), a metric used to measure how fitting and accurate the predicted bounding box and classification is. The YOLO model has a mAP_{50} and $mAP_{50:95}$ of 0.62897 and 0.5415 respectively.

RF-DETR

RF-DETR is the Detection Transformer created by Roboflow. It can be installed from the pip `rfdetr` package. Similar to YOLO, the F-Score and mAP is used to determine the best model.

The model sizes Base, Medium, Large were tested and it was found that Large had the best accuracy. It has an F-Score of 0.8704, mAP_{50} of 0.9239, and $mAP_{50:95}$ of 0.8356.

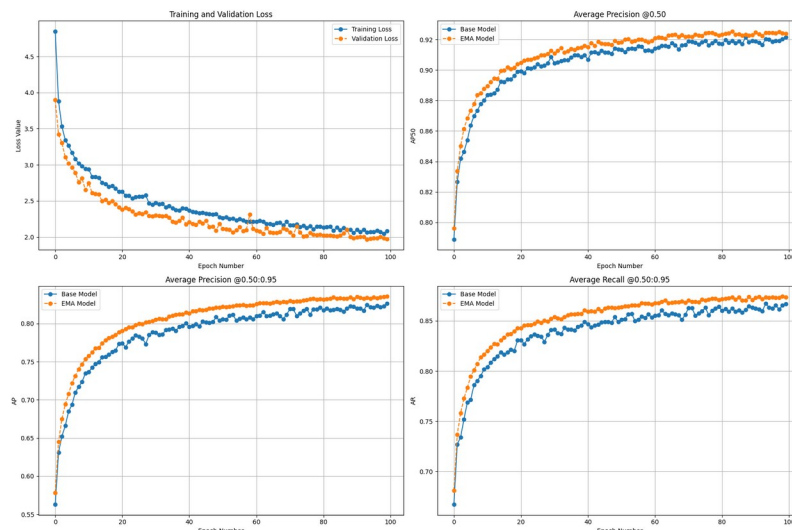


Fig. 4.2.18: Loss curves of RF-DETR Large

However, due to the high complexity and parameters of the Large model, the inference speed was slow at ~4.3s. In contrast, the Medium size used ~0.95s with an F-Score of 0.8488, mAP_{50} of 0.9110, and $mAP_{50:95}$ of 0.8024. In order to maintain real-time inference, the Medium model was selected as it still has relatively high metrics.

4.2.4. Operate

One-Class SVM

For the operation of the One-Class SVM, ten samples are collected at a time (Fig. 4.2.19). Since the model has a binary output (1 for normal, -1 for anomalous), method allows for a confidence value to be generated depending on the number of samples flagged as anomalies.

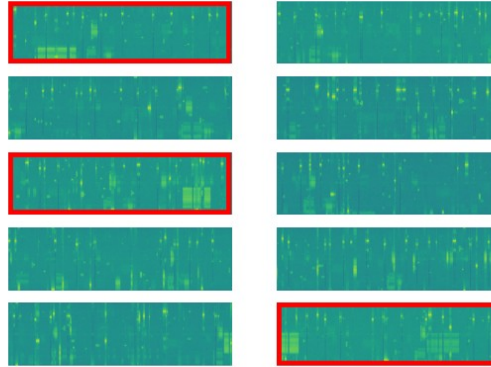


Fig. 4.2.19: Example output of One-Class SVM

A Simple Moving Average (SMA) of the confidence value is also implemented over the last 2 samples. This prevents sudden spikes from triggering a false alarm in the model.

Overall, the model was a decent starting point to explore the field of anomaly detection. However, the model performed poorly. It was susceptible to noisy environments where signals like Bluetooth could trigger the models. Furthermore, it was hard to tune the threshold value, making it hard to adapt for use in different environments. As a result, this model would not be a viable solution for reliable anomaly detection.

Autoencoders

For autoencoders, they are more resilient than the One-Class SVM. However, when changing environments, the reconstruction error still differs slightly. Hence, when first starting up the model, the first 20 samples will be used to find the mean and standard deviation of the errors, assuming that the initial state of the RF spectrum contains no anomalies. Essentially, the statistical method of Z-Score standardization is used to determine the thresholds required. The thresholds can be adjusted using a multiplier on the standard deviation to change how much the error can deviate from the normal before being flagged as an anomaly.

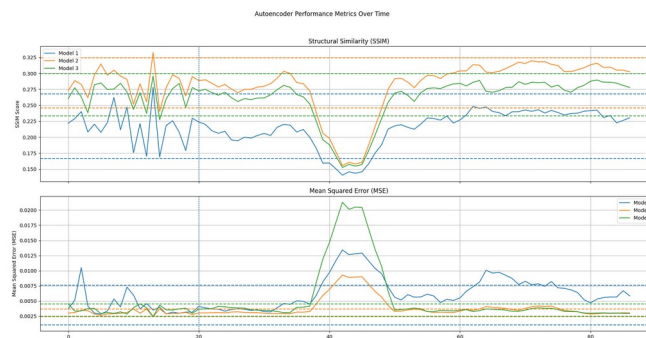


Fig. 4.2.20: Example error (solid) and threshold (dotted) values

Despite being trained using SSIM as the loss function, using both MSE and SSIM to calculate error yields an increase in performance as different types of anomalies affect the metrics in different ways. For example, error calculated with SSIM varies more for signals with smaller bandwidth but high pulse intervals while MSE responds better to signals with a larger bandwidth and duration.

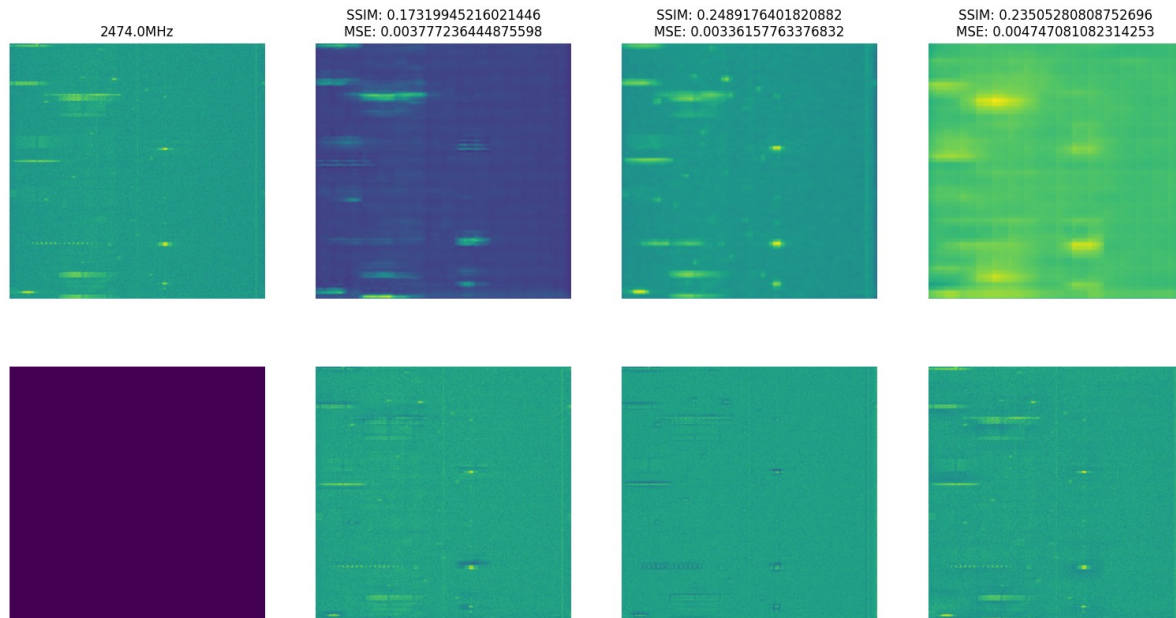


Fig. 4.2.21: Results of Autoencoder4SSIM, Autoencoder3SSIM, Autoencoder1SSIM

Top row: Spectrogram reconstructions; Bottom row: Reconstruction Error

Similar to One-Class SVM, the model implements SMA of both the errors calculated with MSE and SSIM. Unlike the previous SMA, this has a window length of 5, creating more smoothing over the previous signals.

Additionally, the code interfaces with the SDR to scan through the selected frequency band. This allows the full band to be analyzed instead of only the maximum bandwidth allowed by the SDR, helping to identify anomalies which may lie in a small bandwidth.

The three selected models can be used individually or together. If used together, the confidence of detections can be calculated as the percentage of models with positive identifications. Otherwise, the presence of anomalies can be indicated by a singular model only.

Furthermore, the performance of the three selected models (Table 2) occasionally does not match when tested with real-world data, namely the model Autoencoder1SSIM. This is logical as anomalous signals can greatly vary after deployment. In comparison, the test dataset can only contain specific examples of anomalies which the model could be more sensitive to. Hence, while metrics can inform the selection of models to be used, it is best to perform evaluations within the specific environment or site where the models will be deployed.

Object Detection

For both object detection models, the input is preprocessed slightly before being passed to the model. Unlike the synthetic dataset, real-world signals are often noisier and signals need to be separated from the noise floor. Therefore, the Consecutive Mean Excision (CME) algorithm can be used [12]. It has applications in both signal detection as well as suppression of interference, both of which is useful for extracting cleaner signals for object detection. The algorithm is described as follows:

Initialization :

$m = 1$

$J_m = \{\}, I_m = \{1, 2, \dots, N_{FFT}\}$

the sum of magnitude spectrum Φ ,

$$S_m = \sum_{k \in I_m} \Phi(k)$$

The number of indices in set I_m ,

$$N_m = \text{size}(I_m) = N_{FFT}$$

Step 1 Find index sets I_{m+1} and J_{m+1}

$$\begin{cases} I_{m+1} = \{k \in I_m \mid \Phi \geq \frac{TS_m}{N_m} \\ J_{m+1} = \{k \in I_m \mid \Phi < \frac{TS_m}{N_m} \end{cases}$$

Step 2 Calculate number of indices in set J_{m+1} ,

$$P = \text{size}(J_{m+1})$$

Step 3 If $P = 0$ or maximum allowed number of iterations is reached goto Step 8

Step 4 Store the index set J_{m+1}

Step 5 Update $S_1, S_{m+1} = N_m - \sum_{k \in I_{m+1}} \Phi(k)$

Step 6 Update $N, N_{m+1} = N_m - P$

Step 7 $m = m + 1$, Goto step 1

Step 8 Zero out stored indices $J_1 \cup J_2 \cup \dots \cup J_m$ to remove noise

Normally, CME is used by converting time-series signals to the frequency domain to be filtered before using the Inverse Fast-Fourier Transform to return to a cleaner time-series signal. However, in this application, since the object detection models use a spectrogram as input, the CME will be run on each row of the spectrogram, resulting in the spectrogram below:

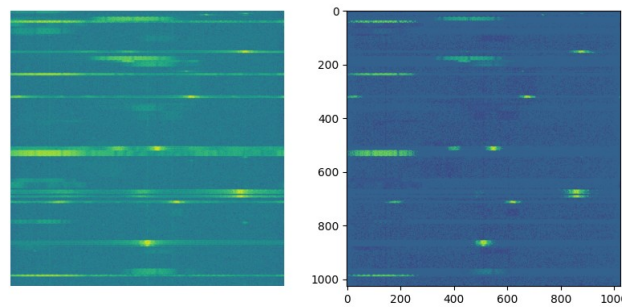


Fig. 4.2.22: Original (left) and Filtered (right) spectrogram

After preprocessing, the spectrogram is converted into a pseudocolour image by applying a colourmap using the OpenCV library which is passed into the models for object detection. Once the known signals are identified, they are removed from the spectrogram using the following method: The median of each frequency bin is subtracted from the identified area, leaving behind a magnitude that looks similar to noise.

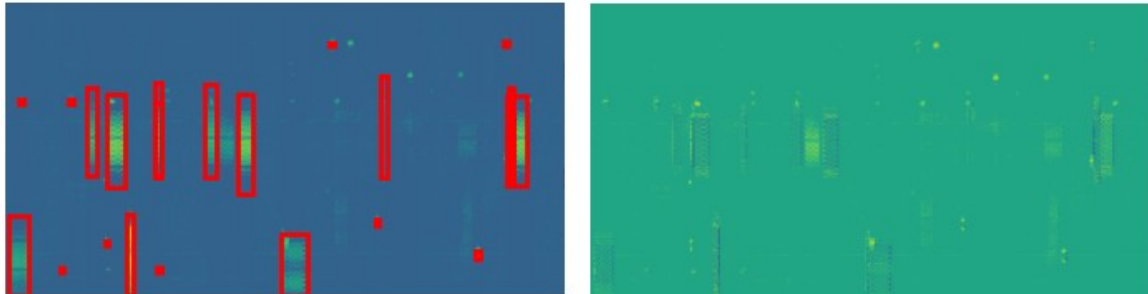


Fig. 4.2.23: Spectrogram with identified (left) and removed signals(right)

After the signals are removed, the arithmetic mean of the spectrogram is calculated in the linear scale and converted back to the logarithmic scale. If this value is higher than the threshold value calculated, it indicates that there are signals still present in the spectrogram which are likely anomalies.

Regarding the threshold, it is similar to the automatic detection of threshold implemented in the autoencoder solution. The inference time of object detection models longer but they are more stable than autoencoders due to the higher complexity and parameters. Hence, the number samples required to detect the threshold is reduced to 10 instead of 20 for a good compromise between threshold selection and timing. Like the One-Class SVM and autoencoder implementations, the solution also uses SMA on the metrics to smooth out spikes.

Similar to autoencoders, both models can be used together or individually. However, the RF-DETR model appears to perform better than the YOLOv11 model, aligning with the results of the metrics. Additionally, both models can be further tuned using adjustable confidence thresholds for individual detected objects.

Challenges

Since this was my first time working with radio frequency data, at the start it was difficult to identify what areas I needed more knowledge on. Ideas like quadrature sampling or the Power Spectral Density were hard to grasp and understand well. However, by reaching out to my mentor, I was able to learn more and better understand the data, as well as signal processing techniques to extract more information.

Additionally, the task of machine learning typically requires a large dataset. Collecting a dataset fit for the models was also challenging as it was hard to identify what was required. The solution was to collect the raw data and build a pipeline which could be easily adjusted and experimented with.

Moreover, tuning the model architecture was time-consuming and tough. The problem lies in the fact that a high model complexity reconstructs both anomalies and normal data, reducing the reconstruction error. On the other hand, if the model is too simple, all inputs will not be

reconstructed properly, leading to a high reconstruction error for everything. This required trial-and-error to determine the best architecture and number of layers, resulting in the current models.

Impact and Conclusion

The models can be used in places which require higher security to expose any entities from attempting to use the RF spectrum to their advantage, as well as detect the presence of any devices transmitting signals, serving to preserve communication channels. Furthermore, if loaded onto a more portable device (i.e. Raspberry Pi), it could allow for convenient use of anomaly detection without a large power source necessary. Applications of such devices is to perform early detection of drones, reducing the chance of drones for intelligence, surveillance, and reconnaissance to remain undetected.

Both autoencoders and object detection methods are suitable for anomaly detection. However, the architecture of autoencoders is close to the limit of performance, unless there is further enhancement in the preprocessing or post-processing stage. In contrast, the object detection models could possibly be improved by expanding the number of classes such that more signals could be identified, though it will be time-consuming to label data. If provided with known anomalous signals, it could function as both an anomaly object detection and novelty detection model.

4.3. Other tasks and projects

Besides the project above, I took part in other projects such as the RF Signal Detector and Broadcast Remote Identification (B-RID) Decoder.

RF Signal Detector

In the RF industry, when determining the presence of high-power signals, in addition to debugging and identifying problems on the physical layer, current solutions are costly (~\$300). Hence, the purpose of the RF Signal Detector is to develop an in-house solution with lower costs, saving money for mass production and use. It uses an RF power detector Integrated Circuit (IC) to detect RF signals.

During this project, I worked with a full-time staff, Jian Qing to create a PCB to house components for both a purely analog and digital version. Jian Qing and I worked together to come up with different solutions and create a circuit diagram for it. While he did most of the routing for the PCB, I verified and alerted him of any possible issues with the circuit and PCB.

Learning RF engineering was a difficult process as both of us previously had little to no knowledge. Moreover, debugging circuitry was challenging due to the fact that the interaction of RF signals with electronics hard to identify. One such complication was the presence of parasitic capacitance, which can reduce signal strength, especially in high-frequency electronics like this case. However, through research and testing, the issue was solved through introducing a ground plane, reducing noise generated by circuit traces. Additionally, the use of a Low-Noise Amplifier was attempted. While successful, due to the cost and reduction of noise through the previous solution, it was deemed unnecessary for the current use case.

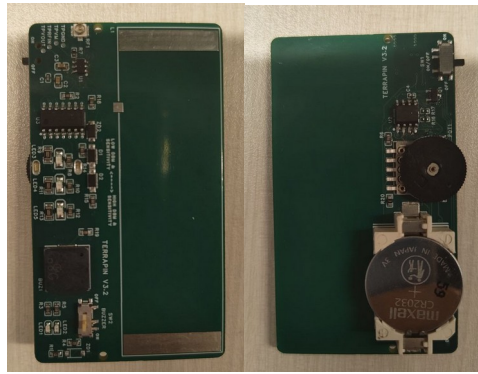


Fig. 4.3.1: Analog version of PCB without antenna attached

After the development of the analog detector, I took the role of main programmer. An ESP32C3 (C3 for short) microcontroller was used due to its small size, allowing for the PCB to remain compact while using the dev board. This prevented the extra developmental cost of moving the microcontroller circuitry to the custom PCB. Furthermore, the C3 has an integrated charge controller, making the charging and discharging of a Lithium Polymer (LiPo) battery convenient. The USB-C connector on the C3 is frequently used and further enhances the convenience of day-to-day use of the detector.

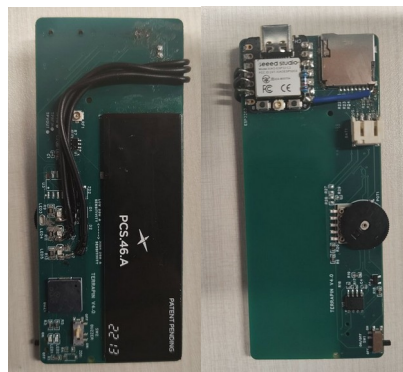


Fig. 4.3.2: Prototype of digital version of PCB

For the programming of the C3, the Arduino IDE was used. The purpose of the microcontroller is to collect RF signal data onto an SD card for further processing. It also replaces certain analog components, such as comparators for the activation of LED indicators, reducing the component costs as well as current draw.

The detectors developed both had good battery duration with 20-50h for the analog version using a 3V coin battery (CR2032) and 80-100h for the digital version using a 3.3V LiPo battery. Both are relatively cheap compared to the market price of SGD 25 and SGD 40 per unit for the analog and digital versions. However, as the production scale increases, the price will reduce per unit, making it even cheaper.

The detectors have been pitched to the SAF for purchase and have received positive feedback on its size and battery duration. This could potentially save the SAF thousands of dollars after mass distribution, compared to the previous, and more costly, alternatives. One practical application of the detectors is to check if radio communication equipment are properly functioning. Moreover, it can be used to identify if high-power signals are present, which has the chance of jamming radio equipment and posing minor health risks over long exposures.

B-RID Decoder

The B-RID Decoder uses an ESP32C6 to detect B-RID signals and decode them. For context, after December 1st, all drones heavier than 250g are required to include a Broadcast Remote Identification transmitter for identification. The purpose of this project is to decoder B-RID signals for detection of drones. Further developments include the transmission and spoofing of B-RID signals which allows the simulation of false signals. This improves the testing capabilities of detection of spoofing by external, and possibly malicious, entities.

During this project, I took a supporting role. I mainly assisted with range tests, testing the largest distance between the decoder and commercially available drone transmitters.



Fig. 4.3.3: Drone capture of me and full-time staff, Zi Iun, after testing

The B-RID protocol uses both Bluetooth Low Energy (BLE), Wi-Fi NAN, and Wi-Fi beacon for transmission. Hence, the ESP32C6 would be the best option as it fits the requirements while having a small footprint, allowing easy integration into other projects where necessary.

From this project, I learn more about how the Bluetooth protocol works, in addition to learning product requirements like minimum range and size limitations. The applications of this is to help Civil Aviation Authority of Singapore (CAAS) to develop B-RID decoders, as well as create anti-spoofing algorithms to prevent malicious agents from flooding detections of drones.

4.4. Skills Required and Company Expectations

Interns require technical skills such as Python and relevant libraries such as Numpy and Matplotlib. Besides the language itself, knowledge is required to set up the proper system environments, being able to resolve any dependency issues or setting up virtual environments should the situation require. A strong foundation in engineering is required to overcome problems that might arise

Apart from technical skills, interns should have a passion and curiosity for engineering. This gives interns the motivation to further perform self-directed learning, taking the effort to educate oneself on any doubts and queries.

Soft skills such as communication and teamwork is also important. There are some situations where interns must work with staff on projects together. To allow quick development and success of the project, both parties need to play their part and assist each other through different tasks.

4.5. Any other Interesting notes about the Internship

Besides interactions with the staff, it was interesting to interact with other interns from different backgrounds such as other polytechnics, JC, and universities. Learning about why they chose to intern at DSTA and talking to them about their background was an enjoyable part of the internship.

5. Reflection and Learning Points

Throughout the internship, I was able to apply Self-Directed Learning to learn many technical skills and build off my foundations. The modules at SP gave me skills in areas like math, electronics, and machine learning which I was able to expand upon for the projects. Such examples include:

- Digital Signal Processing where I applied the Fourier Series and concepts from Engineering Math,
- Anomaly detection machine learning models where I put to use the skills and knowledge learnt about Deep Neural Networks and convolutional layers from Machine Learning and Artificial Intelligence,
- RF Engineering and the routing of the PCB where I applied basics learnt in Electronics from secondary school, the School of Science and Technology, as well as Principles of Electrical and Electronics Engineering.

Besides hard skills, I had the chance to build real-world working experience, being able to attend equipment test sessions and view real products. I also had the opportunity to improve soft skills like communication and collaboration by working with others and assisting in the success of projects like the RF Signal Detector.

Overall, it was an interesting experience where I learnt many skills which can be useful in my future.

5.1. Reflection on CDIO Skills

The CDIO framework was useful in providing a framework and engineering methodology to find a solution to the complex problem of RF anomaly detection.

A critical takeaway was realizing that engineering is an iterative process with trial-and-error. One such example is during the Implement stage, where I initially experimented with the VAE, but these models failed to accurately reconstruct the complex patterns of the RF spectrograms. This prompted me to return to the Design phase, where I tried to preserve the

image hierarchy through the use of convolutional layers, using CVAE in the end. While it was not used as the final model, it showed that there should be a further emphasis on the convolutional based architectures.

This cycle of testing and refining was further enhanced by receiving feedback from my mentors within the C-UAS cluster. Despite the typical goal of maximizing the F-Score, guidance from my supervisor emphasized the operational need of minimizing false positives in a defense context. Hence, I adjusted my evaluation metrics to prioritize high precision, ensuring that the final "Operate" phase would not trigger frequent false alarms in congested urban environments.

This feedback loop allowed me to create models which fit the required problem, taking them one step further for use in real-world systems.

6. Recommendations for the Company

6.1 Learning Points that benefit the company

My investigation into various machine learning architectures for RF anomaly detection revealed that while standard metrics like F-Score are useful, operational success in a defence context depends more on high precision to minimize false positives. Additionally, the different loss functions can be used to signify different types of anomalies, where SSIM is more effective at detecting signals with small bandwidths but high pulse intervals, compared to MSE where it is better for large, sweeping signals. This provides the company with a more sensitive tool for identifying specific drone signatures.

6.2 Recommendations Made and Contributions to the company

I recommend that the output of the autoencoders can be further explored. While it can currently be used as an indication of the type of signals, extra processing can be done to enhance the accuracy of it. Additionally, the development of the RF-DETR Medium model can be continued, as it provides an optimal balance between high accuracy (0.8488 F-Score) and the low latency (~0.95s) required for real-time monitoring.

My work contributed a framework for automatic threshold detection using Z-Score standardization, which allows anomaly detection models to adapt dynamically to the noise floor of different deployment environments. Besides the Machine Learning model, I contributed to the RF signal detector, where it achieved low costs compared to the current market with decent performances in detecting RF signals.

6.3 What can the Company do to remain competitive

To maintain its edge, DSTA should focus on expanding labeled datasets to include a wider variety of known anomalous signals, which would allow the transition from pure novelty detection to more robust classification systems. Furthermore, the company should continue to explore object detection models like RF-DETR, which proved highly effective for feature extraction from complex spectrogram images.

7. Careers in the Industry

The career opportunities in the industry include companies within the Defence Technology Community (DTC), where collaboration with agencies like DSO National Laboratories and CSIT is standard. The industry offers structured career progression from Engineer to Senior Principal roles, providing diverse paths in both technical specialization and programme management.

7.1 My Own Lessons Learnt

Through this internship, I gained a deep understanding of RF engineering concepts, the use of SDRs, and Digital Signal Processing to convert data into meaningful representations like the spectrogram. From the project, it reinforced that engineering is an iterative, trial-and-error process, as demonstrated by testing out different model architectures and seeing which ones are the best fit for the problem at hand.

7.2 Suggestions and recommendations to better prepare students

Future students would benefit from a stronger foundation in Python and machine learning libraries like PyTorch. Additionally, early exposure to signal processing and the hardware-software integration challenges of RF systems, such as parasitic capacitance and noise reduction in PCB design, would better prepare them for the challenge of RF engineering.

8. Conclusion

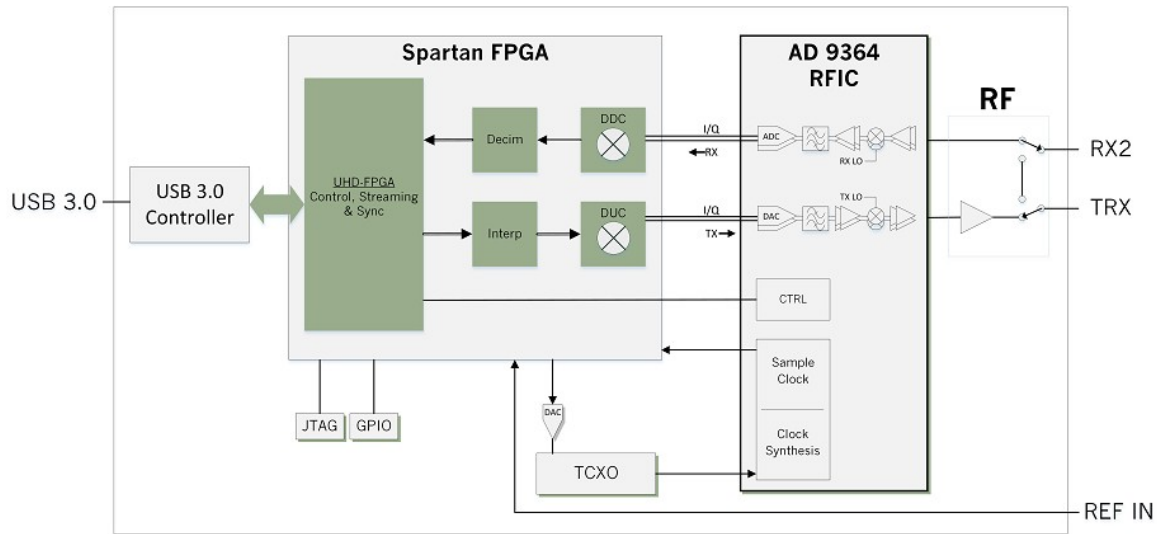
My 22-week internship at the Advanced Systems Programme Centre (ASPC) has been a transformative experience that bridged the gap between academic theory and national security applications. By designing and implementing AI-driven solutions for spectrum sensing and assisting in the physical development of RF signal detectors, I have directly supported the mission of protecting communication channels through the detection of anomalies. This journey has not only enhanced my technical expertise in machine learning and electronics but also fostered the communication and teamwork skills essential for a successful career in the engineering industry.

Overall, this has been an enriching internship which reinforced my interest in computer engineering, while having exposure to a large range of projects. It has been rewarding and instrumental in shaping my understanding of the defence and security of the nation.

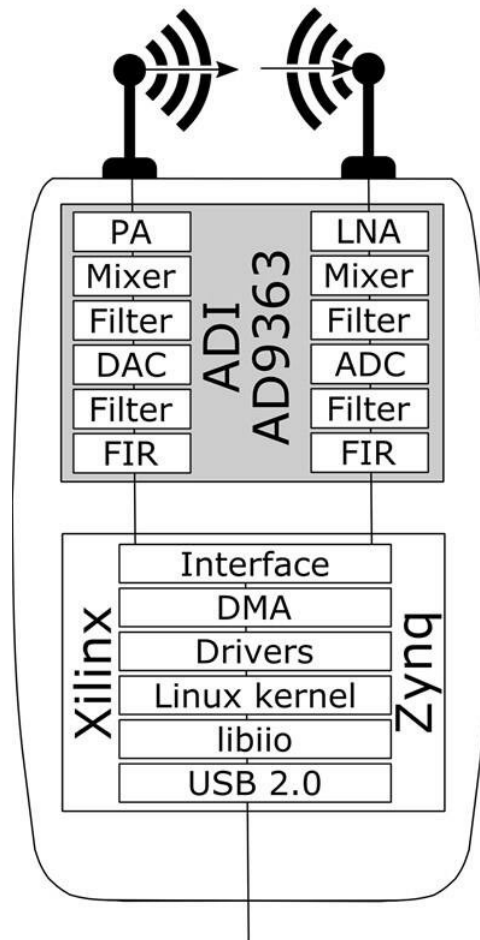
References

- [1] *Defence Science and Technology Agency* (2025) *Wikipedia*. Available at: https://en.wikipedia.org/wiki/Defence_Science_and_Technology_Agency (Accessed: 19 January 2026).
- [2] *About Us* (2025) *DSTA*. Available at: <https://www.dsta.gov.sg/who-we-are/about-us> (Accessed: 19 January 2026).
- [3] *Defence Science and Technology | Ministry of Defence* (2025) *Ministry of Defence*. Available at: <https://www.dtcareers.gov.sg/about-us/> (Accessed: 19 January 2026).
- [4] *With the Community* (2025) *DSTA*. Available at: <https://www.dsta.gov.sg/who-we-are/with-the-community/> (Accessed: 20 January 2026)
- [5] *GeeksforGeeks* (2025) *Understanding one-class support vector machines*, *GeeksforGeeks*. Available at: <https://www.geeksforgeeks.org/machine-learning/understanding-one-class-support-vector-machines/> (Accessed: 21 January 2026).
- [6] *GeeksforGeeks* (2025a) *Support Vector Machine (SVM) algorithm*, *GeeksforGeeks*. Available at: <https://www.geeksforgeeks.org/machine-learning/support-vector-machine-algorithm/> (Accessed: 21 January 2026).
- [7] *A guide to SDR and DSP using python* (2026) *PySDR*. Available at: <https://pysdr.org/content/sampling.html> (Accessed: 25 January 2026).
- [8] *Fast fourier transform* (2026) *Wikipedia*. Available at: https://en.wikipedia.org/wiki/Fast_Fourier_transform (Accessed: 26 January 2026).
- [9] *MATLAB* (2025) *Power spectral density estimates using FFT, MATLAB & Simulink*. Available at: <https://www.mathworks.com/help/signal/ug/power-spectral-density-estimates-using-fft.html> (Accessed: 26 January 2026).
- [10] *Structural similarity index measure* (2025) *Wikipedia*. Available at: https://en.wikipedia.org/wiki/Structural_similarity_index_measure (Accessed: 28 January 2026).
- [11] Wicht, J., Wetzker, U. and Jain, V. (2022) ‘Spectrogram data set for deep-learning-based RF frame detection’, *Data*, 7(12), p. 168. doi:10.3390/data7120168.
- [12] Henttu, P. and Aromaa, S. (2002) ‘Consecutive mean excision algorithm’, *IEEE Seventh International Symposium on Spread Spectrum Techniques and Applications*, 2, pp. 450–454. doi:10.1109/isssta.2002.1048582.

Appendix



Appendix A: Architecture of USRP SDR



Appendix B: Architecture of PlutoSDR

```

class Autoencoder4(nn.Module): # Shape = [1024, 1024]
    def __init__(self, MSE_SSIM, CME=False): # if MSE_SSIM == 1, uses MSE else SSIM
        super(Autoencoder4, self).__init__()

        self.MSE_SSIM = MSE_SSIM
        self.CME = CME
        self.criterion = nn.MSELoss() if (MSE_SSIM == 1) else SSIM(data_range=1)

        self.encoder = nn.Sequential(
            nn.Conv2d(1, 16, 3, stride=1, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(4, stride=4), # 256 x 256
            nn.Conv2d(16, 32, 3, stride=1, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(4, stride=4), # 64 x 64
            nn.Conv2d(32, 64, 3, stride=1, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(4, stride=4), # 16 x 16
        )
        self.decoder = nn.Sequential(
            nn.ConvTranspose2d(64, 32, 5, stride=4,
                               padding=1, output_padding=1), # 16 x 16

            nn.ReLU(),
            nn.ConvTranspose2d(32, 16, 5, stride=4,
                               padding=1, output_padding=1), # 256 x 256

            nn.ReLU(),
            nn.ConvTranspose2d(16, 1, 5, stride=4,
                               padding=1, output_padding=1), # 1024 x 1024
        )

    def forward(self, x):
        x = self.encoder(x)
        x = self.decoder(x)

        if self.MSE_SSIM == 0:
            x = nn.Sigmoid()(x)
        return x

    def loss(self, recon, ground_truth):
        if self.MSE_SSIM == 1:
            return self.criterion(recon, ground_truth)
        else:
            return 1 - self.criterion(recon, ground_truth)

    def norm(self, data_unnorm):
        if self.MSE_SSIM == 1:
            return (data_unnorm - data_unnorm.mean()) / data_unnorm.std()
        else:
            return (data_unnorm - data_unnorm.min()) / (data_unnorm.max() - data_unnorm.min())

```

Appendix C: Autoencoder4 class and implementation